

TEKNOFEST 2026

5G ve Yapay Zekâ ile Akıllı Yol Güvenliği Yarışması

AURA

5G ve Yapay Zekâ ile Akıllı Yol Güvenliği Sistemi

FİNAL TASARIM RAPORU

Takım Adı: **Nankatsu**

Takım ID: _____ (sonra eklenecek)

Başvuru ID: _____ (sonra eklenecek)

Tarih: 28.06.2026

Bu rapordaki tüm doğruluk sayıları, depo ölçüm artefaktlarından otomatik türetilmiş ve yeniden üretilebilirdir (ayrıntı: Bölüm 4.7 Kanıt İzi).

İçindekiler

1	Proje Özeti	3
2	Veri Seti Oluşturulması	3
2.1	Veri Toplama Stratejisi	3
2.2	Etiketleme ve Sınıf Şeması	3
2.3	Veri Dengeleme (Data Balancing)	3
2.4	Veri Artırma (Augmentation)	4
2.5	Train / Val / Test Dağılımı	4
3	Yapay Zekâ Çözümü	4
3.1	Problemin Analizi	4
3.2	Çözüm Mimarisi	5
3.3	Çözüm Detayları	5
4	Çözümün Sınanması	7
4.1	Sinama Protokolü	7
4.2	Tespit Doğruluğu (held-out mAP / P / R / F1)	7
4.3	Davranış Tespiti (P / R / F1)	8
4.4	Plaka Okuma — A/B	8
4.5	QoD Başarım Katkısı (A/B)	9
4.6	İşleme Hızı (FPS)	9
4.7	Hız, Kanıt İzi ve Çözüme Neden Güveniyoruz	10
5	Kaynakça	10

1. Proje Özeti

AURA, yol kenarı trafik kamerası akışından **araç, plaka, hız ve riskli sürücü davranışı** tespiti yapan gerçek bir yapay zekâ çekirdeğini; bu çekirdeği **5G CAMARA Quality-on-Demand (QoD)** ve **Number Verification (NV)** telekom yetenekleriyle birleştiren uçtan uca bir sistemdir. Amaç, yol güvenliğini tehdit eden olayları (dikkatsiz sürüş, araç-içi telefon/sigara kullanımı, hız ihlali) gerçek zamanlı tespit etmek ve **yalnızca kritik anlarda** 5G şebeke kalitesini talep ederek bant genişliğini verimli kullanmaktır.

Proje kapsamında yürütülen başlıca faaliyetler: (i) YOLO26 tabanlı çok-aşamalı bir tespit/takip hattının tasarımı; (ii) sürücü davranışı için landmark kütüphanesi gerektirmeyen, poz-geometrisi ve nesne kanıtını birleştiren hibrit bir motor; (iii) Türk plaka formatına özgü, dürüstlük zırhlarıyla donatılmış bir plaka okuma konsensüs döngüsü; (iv) CAMARA QoD ve NV sözleşmelerini birebir taklit eden mock servisler; (v) çözümün üç gerçek test videosu ve held-out doğrulama setleri üzerinde nicel sınanması. Yapay zekâ çekirdeği (tespit, takip, kararlılık, OCR, hız, risk) **gerçektir**; telekom katmanları gerçek API sözleşmesini taklit eden **mock**'lardır — final ortamında yalnızca uç nokta ve kimlik bilgisi değişir. Kalite güvencesi olarak ~ 780 birim/perf testi, statik denetim (ruff+black) ve sürekli entegrasyon (GitHub Actions) mevcuttur. Held-out ölçümlerde plaka tespiti mAP50 **0,983**, davranış tespiti makro-F1 **1,0** (sıfır çapraz yanlış-pozitif), plaka okuma **3/3 tam-eşleşme** (CER 0,0, sıfır yanlış-onay) ve hedef GPU'da **12–15 FPS** gerçek-zamanlı işleme elde edilmiştir.

2. Veri Seti Oluşturulması

2.1 Veri Toplama Stratejisi

Veri seti, açık-kaynak veri kullanımının serbest olduğu kural çerçevesinde katmanlı bir köprü stratejisiyle oluşturulmuştur. Genel araç/kişi sınıfları (car, bus, truck, motorcycle, person) COCO veri setinden temin edilir. Araç-içi davranış ve plaka sınıfları için kimlik-doğrulaması gerektirmeyen **dört gerçek açık-kaynak bbox seti** indirilip toplanmış ve PIL ile doğrulanmıştır (hepsi CC BY 4.0): license_plate (8.823 görsel), seatbelt (3.104), smoking (557) ve phone (659, sentetik render). İndirilen bu dört set data/processed/ altında işlenmiş olarak bulunur; kaynak/lisans/sınıf-eşlemesi Bölüm 5 kaynakçasında, ek/büyük set çekme planı ise train/datasets.yaml manifestinde belgelidir.

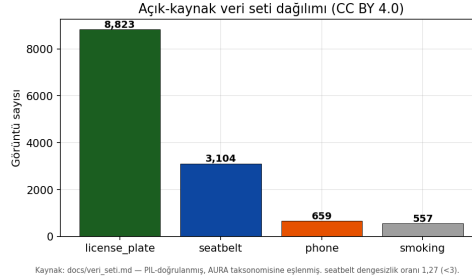
2.2 Etiketleme ve Sınıf Şeması

Etiketler YOLO formatındadır (<class> <cx> <cy> <w> <h>, normalize). Model-uzayı sınıf adları AURA kanonik taksonomisine eşlenir (ör. cell phone → phone, cigarette → smoking) — böylece model değişse de iç sözleşme sabit kalır.

2.3 Veri Dengeleme (Data Balancing)

Sınıf dengesizliği, her bölüm için görüntü sayısı, sınıf-örnek dağılımı ve dengesizlik oranını (en kalabalık/en seyrek sınıf) raporlayan python -m train dataset --report aracıyla nicel ölçülür; oran 3'ü aşarsa araç uyarı üretir. Toplanan seatbelt seti için

ölçülen dengesizlik oranı **1,27**'dir (dengeli; toplam set). Toplanan açık-kaynak setlerin görüntü dağılımı Şekil 1'de verilmiştir. Araç dengesizliği nicel ölçülür ve oran 3'ü aşan seyrek sınıflar için oversampling/hedefli toplama **önerilir**; fiilen uygulanan dengeleme, Ultralytics augmentasyonunun seyrek sınıf lehine güçlendirilmesidir.



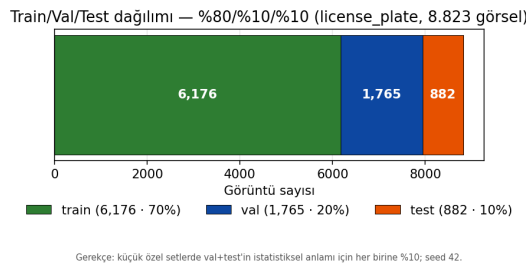
Şekil 1: Toplanan açık-kaynak veri setlerinin görüntü dağılımı (tümü CC BY 4.0).

2.4 Veri Artırma (Augmentation)

Augmentasyon, Ultralytics eğitim hattının yerleşik teknikleriyle uygulanır: **mozaik** (bağlam çeşitliliği ve küçük nesne öğrenimi), **yatay flip** (yön bağımsızlığı), **HSV jitter** (ışık/reng sıcaklığı), **karartma/gamma** (gece ve far-patlama senaryoları) ve **motion blur** (yüksek hızlı araç bulanıklığı). Ablasyon için `--no-augment` ile kapatılabilir. Ek olarak, karanlık kabin görüntülerinde sürücü ROI'sine çalışma anında CLAHE+gamma parlatma uygulanır.

2.5 Train / Val / Test Dağılımı

Veri varsayılan olarak **%80 train / %10 val / %10 test** oranında bölünür (seed 42; Şekil 2). Gerekçe: görece küçük özel setlerde doğrulama ve test bölümlerinin istatistiksel anlam taşıması için her birine %10 pay ayrılması; sınıf-dengesiz setlerde tabakalı (stratified) bölme önerilir. Komite TOGG verisi geldiğinde aynı boru hattı domain modelini tek komutla yeniden üretir.



Şekil 2: license_plate setinin %80/%10/%10 train/val/test dağılımı (8.823 görsel).

3. Yapay Zekâ Çözümü

3.1 Problemin Analizi

Trafik kamerası görüntüsünden tespit, kontrollü laboratuvar koşullarından farklı yapısal zorluklar içerir. AURA, bu zorlukları gerçek footage üzerinde ölçerek tanımlamış ve her

birine hedefli bir tasarım kararıyla yanıt vermiştir (Tablo 1). Bu problemlerin ortak noktası, kamera gürültüsünün ve değişken görüş koşullarının kare-bazlı kararları kararsız kılmasıdır; bu nedenle AURA'nın temel tercihi **kare-merkezli değil ID-merkezli** karar üretmektir — her araç bir takip kimliği (`track_id`) altında izlenir ve tüm kararlar bu kimlik üzerinde zaman içinde biriktirilerek istikrara kavuşturulur.

Tablo 1: Temel problemler ve izlenen çözüm yolları.

Problem	İzlenen çözüm
Karanlık kabin (cam-ardı sürücü)	ROI'de CLAHE + gamma parlatma
Araç tipi titremesi (car↔truck)	Alan-ağırlıklı, track-bazlı sınıf oylaması
Hayalet (phantom) track'ler	<code>min_track_frames</code> çıktı kapısı + IoU dedup
Işık değişimi / hareket bulanıklığı	Augmentasyon (gece/gamma, motion-blur) + sürücü-ROI CLAHE/gamma parlatma
Oklüzyon	ID-merkezli birikim (kısa kayıpta kimlik korunur)
Karanlık plaka il-kodu hatası (3→0/2)	Pozisyon-veto + zemin koşulu → yanlış onay yerine pending
Tek-kare yanlış-pozitif (flicker)	ID-merkezli 16/8 zaman-oylaması

3.2 Çözüm Mimarisi

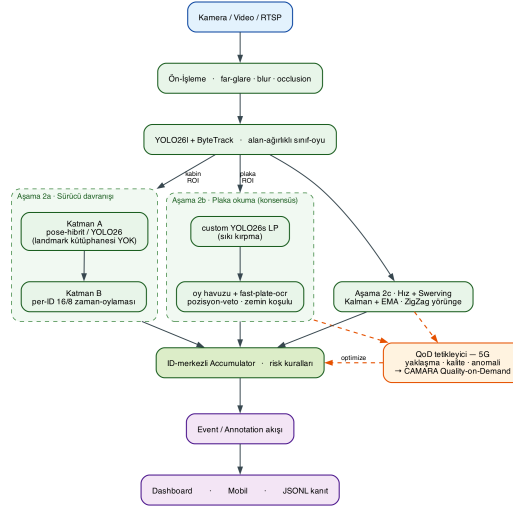
Sistem, ham video akışını etiketli olay/annotation çıktısına dönüştüren bir **kaskad boru hattıdır** (Şekil 3). Ham kare önce ön-işlemeden geçer; Aşama 1 dedektörü (YOLO26 + ByteTrack + alan-ağırlıklı sınıf oyu) araçları tespit/takip eder ve yalnızca **iki ROI** üretir: sürücü kabini ve plaka bölgesi. Bu ROI'ler paralel iki Aşama 2 motoruna gider — sürücü davranışı motoru (Katman A model + Katman B per-ID zaman-oylaması) ve plaka okuma konsensüs döngüsü. Hız/swerving kestirimi ID-merkezli birikim katmanını besler; nihai çıktı olay/annotation akışı olarak dashboard, mobil ve JSONL kanıt izine yayılır. QoD tetikleri (yaklaşma, kalite, anomali) bu akıştan türetilir. (Gerçek↔mock sınırı Bölüm 1'de tanımlanmıştır.)

3.3 Çözüm Detayları

Ön-işleme. Ön-işleme katmanı far-patlama, motion-blur, yansıma ve occlusion için config'ten aç/kapanabilen filtre kancaları sunar; mevcut sürümde fiilen uygulanan iyileştirme, OCR ve sürücü-durumu doğruluğunu artıran **ROI-düzeyi** işlemedir: plaka/sürücü kırığına CLAHE + gamma parlatma ve küçük kırık için süper-çözünürlük (gece/los koşul telafisi).

Aşama 1 — Tespit ve Takip. Ana dedektör varsayılan olarak doğruluk-önce stok `yolo26l` (Ultralytics 8.4, en güncel YOLO ailesi); ByteTrack her araca benzersiz kimlik atar. Aynı aracın kareler arası sınıf salınımı (uzaktan `truck`, yakından `car`), track başına **alan-ağırlıklı sınıf oylaması** ($\text{güven} \times \text{bbox_alan/kare_alan}$) ile çözülür.

Aşama 2a — Sürücü Davranışı (iki katman). Sürücü ROI'si önce kişi kutusuna



Şekil 3: AURA kuşbakışı kaskad boru hattı (kaynak: `aura/pipeline/pipeline.py`, `docs/diagrams/`).

daraltılır. Katman A iki backend'den birini kullanır: fine-tune YOLO26 detection veya — fine-tune gerektirmeyen — **YOLO26-pose** keypoint geometrisi (bilek↔ağız/kulak görelî yakınlığı, yüz-genişliği biriminde, ölçek-bağımsız), nesne kanıtıyla hibrit güçlendirilir; landmark/MediaPipe kütüphanesi **kullanılmaz**. Katman B (DriverStateEngine) her track için zaman-oylaması yürütür: bir bayrak son 16 karenin en az 8'inde doğruysa aktif sayılır (tek-kare yanlış-pozitif elenir).

Aşama 2b — Plaka Okuma ve Konsensüs. Araç sweet-spot'a girince OCR etkinleşir; **özel eğitilmiş YOLO26s LP dedektörü** (`custom_license_plate`, held-out mAP50 0,983) plakayı sıkı kırpar. Plakaya-özel hafif ONNX motoru **fast-plate-ocr** (varsayılan; kurulu değilse `EasyOCR` fallback) okur. Her okuma, OCR güveni $\times$ kaynak kalitesi (LP kırpık yüksekliği) ile ağırlıklanıp track ömrü boyunca **kalıcı oy havuzunda** biriktirilir. Türk plaka formatına (`~\d{2}[A-Z]{1,3}\d{2,4}$`) normalize edilen okumalar pozisyon-hızlı karakter füzyonuyla birleştirilir; onay için her pozisyonda kazanan karakter ikinciye mutlak bir marjla geçmek zorundadır, aksi halde karar dürüstçe `pending`'e çevrilir — **yanlış plaka asla kesinleştirilmez**.

Stabilite/Doğruluk Zırhları (K-004, config-driven). (i) Kayan-karakter onay marjı (`confirm_min_char_margin=2.0`): pozisyon belirsizse plaka onaylanmaz, `pending` denir. (ii) Takip/phantom çıktı kapısı (`track_id=-1, min_output_frames`): kimliğe bağlanmamış hayalet tespitler olay üretmez. (iii) ROI alan tavanı (`max_roi_area_ratio=0.10`): anormal büyük ROI'ler kırılır. Hiçbir eşik videoya-özel sabit içermez (oran-bazlı, ölçek-bağımsız).

Hız ve Swerving. Hız kalibrasyon-bağımlıdır (tripwire/ipm/metric, Kalman+EMA); kalibrasyon yoksa sistem mutlak hız iddia etmez, yalnız görelî-hız bayrağı üretir. Swerving, aracın merkez-x serisindeki ZigZag ekstremum sayımıyla (araç-genişliği birimi, saniye penceresi) kalibrasyonsuz tespit edilir.

Yazılım ve Donanım. Python 3.13, PyTorch (Ultralytics 8.4.66), `fast-plate-`

ocr/EasyOCR, OpenCV, Shapely, FastAPI/Uvicorn. Cihaz otomatik seçilir (CUDA → MPS → CPU). Geliştirme Apple Silicon/MPS; hedef sunucu dağıtımı NVIDIA CUDA üzerindedir — gerçek ölçüm donanımı RTX 5070 Laptop GPU (4.608 CUDA çekirdeği, 8 GB VRAM, Compute 12.0/Blackwell, torch 2.8.0+cu128; bkz. Bölüm 4.6).

4. Çözümün Sınanması

4.1 Sınama Protokolü

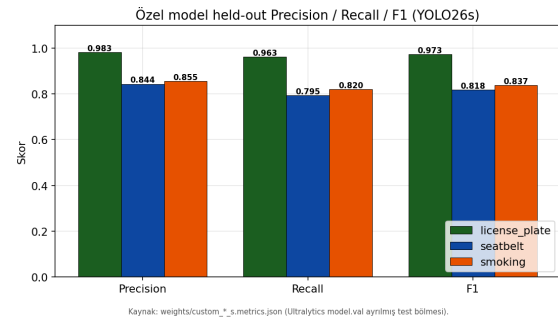
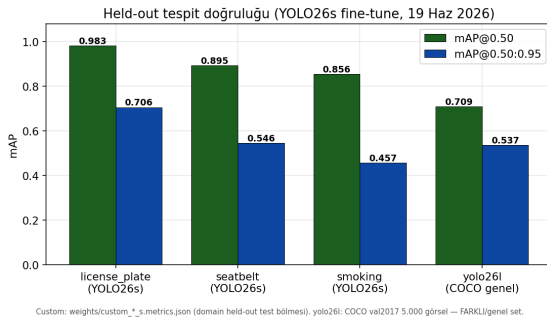
Çözüm üç tamamlayıcı düzeyde sınanmıştır: (i) etiketli **held-out doğrulama setleri** üzerinde istatistiksel tespit doğruluğu (mAP/P/R; Bölüm 4.2); (ii) üç gerçek test videosu (kapalı otopark, TOGG; GT plaka 34TC8532) üzerinde davranış P/R/F1, plaka exact-match (CER ile), FPS (Bölüm 4.3–4.6); (iii) QoD’nin başarımlar katkısı (A/B), kare-düzeyi GT içeren kontrollü sentetik set üzerinde (Bölüm 4.5). Şartnamenin 4.5 maddesi gereği (“kanıtlanamayan hedef puanlanmaz”), rapordaki her sayı bir komuta ve artefakta bağlıdır (Bölüm 4.7).

4.2 Tespit Doğruluğu (held-out mAP / P / R / F1)

Özel sınıfların YOLO26s fine-tune eğitimi **tamamlanmıştır**; ayrılmış test bölmesinde ölçülen held-out doğrulukları Tablo 2 ile Şekil 4’de verilmiştir. Varsayılan stok yolo26l ayrıca COCO val2017 (5.000 görsel) held-out üzerinde değerlendirilmiştir (genel doğruluk göstergesi). Telefon (phone) tespiti stok yolo26l’in COCO “cell phone” sınıfıyla yapılır; ayrı özel held-out ölçümü yoktur (davranış doğruluğu Tablo 3’de video-düzeyinde verilmiştir).

Tablo 2: Held-out tespit doğruluğu (özel modeller: weights/custom_*_s.metrics.json; stok: COCO val2017). [†] license_plate üretimde varsayılan LP dedektörüdür; seatbelt/smoking fine-tune held-out sonuçlarıdır (üretim hattında opsiyonel/devre-dışı).

Model / set	mAP50	mAP50-95	Precision	Recall	F1
license_plate (yolo26s)	0,983	0,706	0,982	0,963	0,973
seatbelt (yolo26s) [†]	0,895	0,546	0,844	0,795	0,818
smoking (yolo26s) [†]	0,856	0,457	0,855	0,820	0,837
yolo26l — COCO val2017 (genel)	0,709	0,537	0,740	0,641	—



Şekil 4: Sol: held-out mAP50 / mAP50-95. Sağ: özel modellerin held-out Precision / Recall / F1 (yolo26s). Stok yolo26l COCO genel settir.

4.3 Davranış Tespiti (P / R / F1)

Üç gerçek video üzerinde video-düzeyi davranış tespiti iki dedektör profiliyle ölçülmüştür (Tablo 3). **Her iki dedektör de** çapraz yanlış-pozitif üretmez (makro-F1 1,0; phone/smoking/swerving için P=R=F1=1,0); araç sınıfı doğruluğu (accuracy) **%100**'dür. *Dürüst bağlam*: sınıf başına destek N=1 (üç videoda birer pozitif örnek); istatistiksel güç held-out mAP setlerinden (Tablo 2) gelir, video seti yalnız uçtan-uca doğrulamadır. Emniyet kemeri (seatbelt) yalnızca held-out mAP (0,895) ile sınanmıştır; üç videoda kemer-GT örneği bulunmadığından video-düzeyi P/R/F1 raporlanmamıştır.

Tablo 3: Davranış tespiti video-düzeyi P/R/F1 (kaynak: eval_results/metrics_report.md).

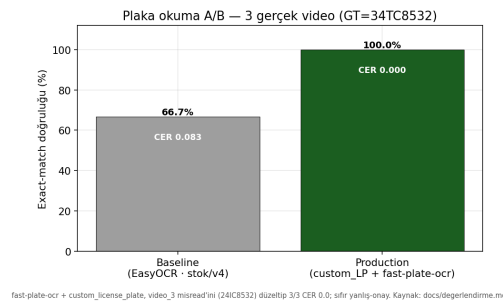
Dedektör	phone	smoking	swerving	Makro F1
yolo26l (stok, varsayılan)	1,0/1,0/1,0	1,0/1,0/1,0	1,0/1,0/1,0	1,00
v4-finetune (A/B kıyas tabanı)	1,0/1,0/1,0	1,0/1,0/1,0	1,0/1,0/1,0	1,00

4.4 Plaka Okuma — A/B

Plaka okuma, footage'ın zorlu (karanlık otopark) koşulları nedeniyle en kritik dürüstlük sınavıdır. Baseline OCR (EasyOCR) ile production hattı (özel YOLO26s LP + fast-plate-ocr) arasındaki A/B Tablo 4 ve Şekil 5'te verilmiştir. fast-plate-ocr, baseline'in video_3'teki sistematik il-kodu misread'ini (24IC8532) düzelterek sonucu **3/3 exact-match, CER 0,0**'a çekmiş ve video_1/video_2'yi korumuştur; her iki hatta da yanlış-onay sıfırdır.

Tablo 4: Plaka okuma A/B — 3 gerçek video, GT=34TC8532 (kaynak: docs/degerlendirme.md).

Hat	Exact-match	CER	Confirmed	Partial	Yanlış-onay
Baseline (EasyOCR · stok/v4)	2/3 (%66,7)	0,083	2	1	0
Production (custom_LP + fast-plate-ocr)	3/3 (%100)	0,0	3	0	0



Şekil 5: Plaka exact-match A/B: baseline 2/3 (CER 0,083) → production 3/3 (CER 0,0).

En kritik tasarım garantisi: sistem belirsiz/uzak/bulanık bir okumayı **asla yanlış plaka olarak kesinleştirmez**; yakın-net video doğru plakayı CONFIRMED verirken belirsiz

okuma onurlu bir `pending`'tir. Bu davranış Bölüm 3.3'teki onay marjı, pozisyon-veto ve zemin koşulu zırhlarının ölçülen sonucudur.

4.5 QoD Başarım Katkısı (A/B)

QoD entegrasyonu, A/B harness ile **yeniden-üretilebilir** biçimde ölçülür: aynı kaynak, QoD OFF (düşük çözünürlük/bant simülasyonu) ve QoD ON (tam çözünürlük) ile koşulup plaka, küçük-nesne ve tespit metrikleri kıyaslanır (`aura.eval --qod-comparison`). **Dürüst not (kritik):** mevcut kontrollü koşuda (`eval_results/report.json`) OFF baseline'ı zaten yeterli kalitede olduğundan ölçülen delta ≈ 0 'dır. Pozitif delta yalnızca OFF senaryosu bant/çözünürlük baskısı altındayken (küçük/uzak plaka ROI'si okunamaz piksele düştüğünde) ortaya çıkar ve daha agresif bir OFF parametresiyle yeniden üretilir (örn. `docs/degerlendirme.md`: plaka +33pp). Dolayısıyla QoD'nin katkısı, sabit ölçülmüş tek bir sayı değil, **mekanizmanın yönlülüğüdür**: `vehicle_approach` tetiği şartnamenin "TOGG yaklaşıncı QoD" senaryosunu birebir karşılayıp kritik anda kaliteyi talep eder; harness bu deltayı tek komutla ölçmeye hazırdır.

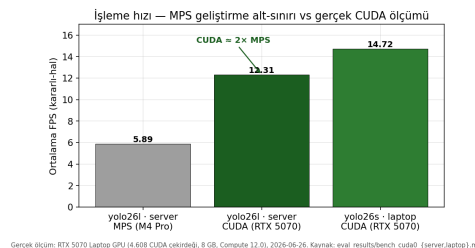
4.6 İşleme Hızı (FPS)

İşleme hızı hem geliştirme ortamında (Apple Silicon/MPS, M4 Pro) hem de hedef sunucu donanımında (NVIDIA RTX 5070 Laptop GPU, **4.608 CUDA çekirdeği**) **gerçek ölçümle** değerlendirilmiştir (Tablo 5, Şekil 6; `eval_results/bench_cuda0_*.md`, 2026-06-26). CUDA, MPS geliştirme alt-sınırının (`yolo261` $\sim 5,9$ FPS) $\approx 2\times$ 'i iş-hacmi sağlar.

Tablo 5: İşleme hızı — gerçek ölçüm (`eval_results/bench_cuda0_*.md`, 2026-06-26).

Profil · dedektör · imgsz	Ortalama FPS	p50 (ms)	p95 (ms)
server · yolo261 · 960 (CUDA)	12,31	80	93
laptop · yolo26s · 640 (CUDA)	14,72	65	88
server · yolo261 (MPS, M4 Pro)	$\sim 5,9$	—	—

Kare-başına **gecikme** ($p95=93$ ms) ile **iş-hacmi** (~ 12 FPS) ayrı büyüklüklerdir: sistem ~ 12 FPS iş-hacmiyle 25–30 FPS'lik bir kamera akışını kare-örnekleme (her 2–3 karede bir) ile olay-tespiti amacıyla işler; p95 gecikme zarfı bu kullanım için yeterlidir.



Şekil 6: İşleme hızı: MPS geliştirme alt-sınırı vs gerçek CUDA ölçümü (RTX 5070 Laptop GPU, 4.608 çekirdek, Compute 12.0).

4.7 Hız, Kanıt İzi ve Çözüme Neden Güveniyoruz

Hız/swerving. Mutlak hız bu raporda **nicel olarak sinanmamıştır** (MAE/MAPE harness'ı hazır; kalibrasyon ve gerçek-hız GT geldiğinde nicel hata üretir, yoksa dürüstçe mutlak hız iddia edilmez). Sinanan yetenek kalibrasyonsuz swerving'dir: üç videodan video_3'te gerçekten tespit edilmiş ve davranış makro-F1'ine dahildir (Bölüm 4.3).

Kanıt izi (şartname 4.5). Her çıktının makineyle üretildiği üç artefaktla kanıtlanır: (1) zaman-damgalı JSONL olay izi (--save-events); (2) annotated mp4 + JSON oy dökümü (tools/test_video.py); (3) türetilebilir metrik raporu (aura.eval --metrics-report) ki Bölüm 4 tablolarını bu özetlerden yeniden üretir — yani rapordaki hiçbir sayı elle girilmemiştir. Ayrıca ~780 testin bir bölümü CAMARA QoD/NV mock sözleşme uyumunu (sessiz doğrulama, oturum yaşam-döngüsü) doğrular.

Çözüme neden güveniyoruz? Güven, ölçülen sayıların ötesinde **yapısal** gerekçelere dayanır: tüm eşikler oran/boyut-temelli ve ölçek-bağımsızdır (videoya-özel sabit yok, K-004), dolayısıyla sonuçlar tek çekime aşırı-uyumlanmaz; belirsizlikte sistem yanlış üretmek yerine dürüstçe çekimser kalır (plakada **sıfır yanlış-onay**) ve bu güvenceler config-driven, gerçek videoda doğrulanmış zırlara (Bölüm 3.3) dayanır. Sinama kümesinin sınırı (üç-videoluk küçük set; istatistiksel mAP held-out sette ayrıca ölçülmüştür, Tablo 2) açıkça belirtilmiş olup komite verisiyle güçlendirilecektir.

5. Kaynakça

Başlıca kaynaklar (tam liste — 48 kaynak — açık veri lisansları ve izlenebilirlik tablosu: kaynakca.md):

1. Ultralytics, *Ultralytics YOLO Documentation* (v8.4.66, YOLO26), 2026. <https://docs.ultralytics.com> (erişim 2026).
2. Y. Zhang vd., *ByteTrack: Multi-Object Tracking by Associating Every Detection Box*, ECCV, 2022. arXiv:2110.06864.
3. T.-Y. Lin vd., *Microsoft COCO: Common Objects in Context*, ECCV, 2014. arXiv:1405.0312. (CC BY 4.0)
4. ankandrew, *fast-plate-ocr*, 2024–2026. <https://github.com/ankandrew/fast-plate-ocr>.
5. JaidedAI, *EasyOCR*, 2020–2026. <https://github.com/JaidedAI/EasyOCR>. (Apache-2.0)
6. K. Zuiderveld, *Contrast Limited Adaptive Histogram Equalization (CLAHE)*, Graphics Gems IV, 1994 · R. E. Kalman, *A New Approach to Linear Filtering*, ASME J. Basic Eng., 1960. (ön-işleme / hız)
7. CAMARA Project (Linux Foundation), *Quality-on-Demand API & Number Verification API*, 2023–2026. <https://github.com/camaraproject>.
8. 3GPP, *System Architecture for the 5G System*, TS 23.501, Rel-18. <https://www.3gpp.org>.
9. keremberke, *License Plate Object Detection*, Hugging Face, 2023. (CC BY 4.0) — license_plate 8.823 görsel.
10. ramankamran, *Seatbelt Detection (YOLOv11)*, Hugging Face, 2024. (CC BY 4.0) — seatbelt 3.104 görsel.
11. *CigDet — Cigarette Detection*, Mendeley Data, 2021. DOI:10.17632/6hyrr8typ7.1. (CC BY 4.0) — smoking 557 görsel.
12. anywaylabs, *Synthetic Driver Monitoring*, Hugging Face, 2024. (CC BY 4.0) — phone 659 görsel.
13. M. Everingham vd., *The PASCAL VOC Challenge* (mAP metodolojisi), IJCV, 2010.
14. Yapay zekâ desteği: raporun yazım/düzenlenmesinde büyük dil modelinden (Anthropic Claude) destek alınmıştır; tüm teknik içerik ve sayılar takımca üretilip depo artefaktlarından doğrulanmıştır.